

A Static Approach to Prioritizing JUnit Test Cases

Hong Mei, *Senior Member, IEEE*, Dan Hao, Lingming Zhang, *Student Member, IEEE*,
Lu Zhang, Ji Zhou, and Gregg Rothmel, *Member, IEEE Computer Society*

Abstract—Test case prioritization is used in regression testing to schedule the execution order of test cases so as to expose faults earlier in testing. Over the past few years, many test case prioritization techniques have been proposed in the literature. Most of these techniques require data on dynamic execution in the form of code coverage information for test cases. However, the collection of dynamic code coverage information on test cases has several associated drawbacks including cost increases and reduction in prioritization precision. In this paper, we propose an approach to prioritizing test cases in the absence of coverage information that operates on Java programs tested under the JUnit framework—an increasingly popular class of systems. Our approach, **JUnit test case Prioritization Techniques operating in the Absence of coverage information (JUPTA)**, analyzes the static call graphs of JUnit test cases and the program under test to estimate the ability of each test case to achieve code coverage, and then schedules the order of these test cases based on those estimates. To evaluate the effectiveness of JUPTA, we conducted an empirical study on 19 versions of four Java programs ranging from 2K-80K lines of code, and compared several variants of JUPTA with three control techniques, and several other existing dynamic coverage-based test case prioritization techniques, assessing the abilities of the techniques to increase the rate of fault detection of test suites. Our results show that the test suites constructed by JUPTA are more effective than those in random and untreated test orders in terms of fault-detection effectiveness. Although the test suites constructed by dynamic coverage-based techniques retain fault-detection effectiveness advantages, the fault-detection effectiveness of the test suites constructed by JUPTA is close to that of the test suites constructed by those techniques, and the fault-detection effectiveness of the test suites constructed by some of JUPTA's variants is better than that of the test suites constructed by several of those techniques.

Index Terms—Software testing, regression testing, test case prioritization, JUnit, call graph

1 INTRODUCTION

REGRESSION testing is an expensive testing process that has been estimated to account for almost half of the cost of software maintenance [1], [2], [3]. To improve the regression testing process, test case prioritization techniques (e.g., [4], [5], [6], [7]; Section 5 provides a summary) schedule the execution order of test cases so as to meet some goal, such as an increased rate of fault detection or a reduction in missed faults when test suites cannot be run to completion.

Most test case prioritization techniques [1], [7], [8] proposed to date rely on the use of dynamically collected code coverage information (e.g., statement or branch

coverage) obtained through instrumentation and execution of the system under test. Typically, these techniques prioritize test cases based on the amount of coverage they achieve. While several of these techniques have been shown to be effective (see, e.g., [8]), their reliance on dynamically collected coverage information can also pose problems:

- H. Mei, D. Hao, and Lu Zhang are with the School of Electronics Engineering and Computer Science, Institute of Software, Peking University, Beijing 100871, PR China and the Key Laboratory of High Confidence Software Technologies, Ministry of Education, PR China. E-mail: meih@pku.edu.cn, {haod, zhanglu}@sei.pku.edu.cn.
- Lingming Zhang is with the Center for Advanced Research in Software Engineering (ARISE), Department of Electrical and Computer Engineering, The University of Texas at Austin, ACES 5.132, C5000, 201 E 24th Street, Austin, TX 78712-0240. E-mail: zhanglm10@gmail.com.
- J. Zhou is with Baidu.com, Inc., Baidu Building, No. 10 Shangdi Street, Haidian District, Beijing 100871, PR China. E-mail: zhouji@baidu.com.
- G. Rothmel is with the Department of Computer Science and Engineering, University of Nebraska-Lincoln, Lincoln, NE 68588 and the Division of Web Science and Technology, Korea Advanced Institute of Science and Technology, Taejeon, South Korea. E-mail: grother@cse.unl.edu.

Manuscript received 26 Nov. 2010; revised 15 Sept. 2011; accepted 2 Oct. 2011; published online 5 Oct. 2011.

Recommended for acceptance by A. Bertolino.

For information on obtaining reprints of this article, please send e-mail to: tse@computer.org, and reference IEEECS Log Number TSE-2010-11-0352.

Digital Object Identifier no. 10.1109/TSE.2011.106.

- While techniques [9], [10], [11] for profiling programs continue to improve, they still do add instrumentation, execution, and storage overhead costs to the testing process. Furthermore, even efficient instrumentation can perturb the behavior of certain types of software systems, such as real-time systems with performance bounds or systems engaged in high-performance computational activities. This can result in the collection of coverage data that does not accurately reflect the program's execution path in the cases where the instrumentation is not present, and such inaccuracies in execution information can in turn result in inaccurate prioritization.
- Code-coverage-based prioritization techniques [5], [12], [13] typically rely on coverage data obtained on a *prior* version of a system to order tests for use on a *subsequent* version. Such coverage data are essentially used to *estimate* test case coverage on the subsequent version. In some cases, however, code modifications between versions can cause many test cases to behave differently on subsequent versions, rendering the estimate obtained from prior coverage data imprecise [14]. This too can affect the accuracy of prioritization [15], [16].

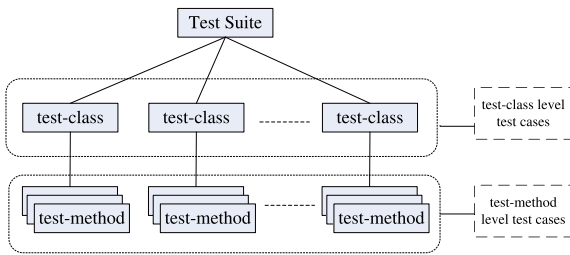


Fig. 1. JUnit test suite structure.

- New test cases for which coverage data have not yet been collected cannot be easily added into prioritized orders. Thus, when test suites are augmented to cover newly added code or system components, it is not clear how to integrate their use into a regression testing process.

Regression testing processes that involve short maintenance and testing cycles can exacerbate some of the foregoing problems. For example, in “nightly-build-and-test” scenarios, the likelihood of being able to update coverage information on all test cases affected by code changes decreases, in turn increasing the likelihood that the code coverage information used to prioritize test cases will have decayed and that prioritization will be inaccurate.

In this work, we present a static approach to prioritizing test cases that avoids the foregoing drawbacks. Our approach, JUnit test case Prioritization Techniques operating in the Absence of coverage information (JUPTA), differs from existing dynamic coverage-based test case prioritization techniques in that it prioritizes test cases based on coverage information estimated from static structures rather than gathered through instrumentation and execution.

JUPTA is defined to operate on Java programs tested in the JUnit framework,¹ which is widely used in developing Java software. A JUnit test case is a piece of executable source code containing testing content (i.e., a sequence of method invocations). Thus, we can determine which methods each JUnit test case invokes and use such information to create static call graphs for JUnit test cases. We can use these call graphs to estimate test coverage and through this to prioritize JUnit test cases.²

While our prior work [17] on JUPTA deals only with JUnit test cases at the test-method level, JUPTA can be used to prioritize test cases either at the test-method level or at the test-class level since JUnit has two levels of test cases (see Fig. 1). This is important because test-class level JUnit tests are used in integration testing of Java systems and regression test suites often are composed of such integration tests. Moreover, similar to existing code-based test case prioritization techniques, JUPTA can be implemented based on either an initial one-time ordering strategy or a strategy in which feedback is employed after each test case is placed in prioritized order to improve subsequent orderings.

To evaluate the effectiveness of JUPTA, we conducted an empirical study on 19 versions of four Java programs ranging from 2K-80K lines of code that have been widely used in prior studies of software testing and analysis techniques (e.g., [18], [19], [20]). In the study, we examine

several prioritization techniques, including several of the most common and widely studied dynamic coverage-based techniques, and three control techniques (a random ordering technique, the use of untreated test cases, and an optimal technique). We compare these to several variants of the JUPTA approach. Our results show that some of the JUPTA variants perform quite well in comparison to dynamic coverage-based prioritization techniques. As dynamic coverage-based prioritization techniques use actual coverage information,³ these techniques are expected to generate test suites with higher fault-detection effectiveness than JUPTA, which uses estimated coverage information. However, according to the experimental results, two variants of JUPTA are statistically significantly equivalent in fault-detection effectiveness to the best dynamic coverage-based techniques. Our results also show that JUPTA is usually better than untreated and randomly ordered test suites, while (as expected) not achieving the performance of the optimal approach. Finally, our results help us distinguish between variants of the JUPTA approach; for example, we find that JUPTA is most effective when it is implemented to use feedback and to operate at the test-method level.

The remainder of this paper is organized as follows: Section 2 provides background information. Section 3 presents our JUPTA approach and its variants. Section 4 presents the empirical study. Section 5 reviews related work and Section 6 concludes.

2 BACKGROUND

The test case prioritization problem is formally defined in [1] as follows. Suppose that PT is the set of permutations on the test cases in a given test suite T , the test case prioritization problem is to find $PS \in PT$ such that $\forall T' \in PT$, $f(PS) \geq f(T')$, where f is a function from PT to the real numbers which represent the rate of fault detection.

A wide range of prioritization techniques have been proposed and empirically studied (e.g., [1], [5], [7], [8], [21], [22], [23], [24], [25]). As noted in Section 1, most of these techniques utilize some form of dynamic code coverage information to direct the prioritization effort. Given code coverage information for a test suite T , one way to prioritize is to use a “total-coverage” prioritization technique that orders test cases in terms of the total number of code components that they cover. Total-coverage techniques can be improved by adding *feedback*, using an iterative greedy approach in which each “next” test case is placed in the prioritized order taking into account the effect of test cases already placed in the order. For example, an “additional-block-coverage” technique prioritizes test cases in terms of the numbers of new (not-yet-covered) blocks they cover by iteratively selecting the test case that covers the most not-yet-covered blocks until all blocks are covered, then repeating this process until all test cases have been prioritized.

3. Traditional dynamic coverage-based prioritization techniques use coverage information on the prior version of a system, but dynamic coverage-based prioritization techniques studied in the empirical study of this paper use coverage information on the new version. Therefore, dynamic techniques in this paper might be better than traditional dynamic techniques in fault-detection effectiveness.

1. <http://www.junit.org>.

2. Thus, JUPTA could function more broadly on any system for which test cases are implemented as code that calls tested subsystems.

Figure 2 consists of two parts, (a) and (b), showing Java code. Part (a) shows the `BasicAccount` and `Account` classes. `BasicAccount` has private fields `thisAcnt` and `balance`, and public methods `BasicAccount(String acnt, double amt)`, `getBalance()`, and `setBalance(double balance)`. `Account` extends `BasicAccount` and has public methods `Account(String acnt, double amt)`, `sendto(String account)`, `deposit(double amt)`, `withdraw(double amt)`, and `transfer(double amt, String anotherAcnt)`. Part (b) shows the `Testcases` class with methods `setUp()`, `tearDown()`, and six test methods: `test1()`, `test2()`, `test3()`, `test4()`, `test5()`, and `test6()`. Annotations in part (b) include a box pointing to `setUp()` and `tearDown()` labeled "preprocess and postprocess for test method execution", and another box pointing to `test1()` through `test6()` labeled "Six test methods to be prioritized. We refer to them as test cases T1, T2, T3, T4, T5, and T6 in this paper."

```

public class BasicAccount {
private String thisAcnt;
private double balance;
public BasicAccount(String acnt, double amt) {
    this.balance = amt;
    this.thisAcnt = acnt;
public double getBalance() {
    return this.balance;
public void setBalance(double balance) {
    this.balance = balance;
}

public class Account extends BasicAccount{
public Account(String acnt, double amt) {
    super(acnt, amt);
public void sendto(String account) {
    // send money to target account
public boolean deposit(double amt) {
    if (amt > 0) {
        setBalance(getBalance() + amt);
        return true;
    }
    else
        return false;
public boolean withdraw(double amt) {
    if (getBalance() >= amt && amt > 0) {
        setBalance(getBalance() - amt);
        return true;
    }
    else
        return false;
public boolean transfer(double amt, String anotherAcnt) {
    double fee = amt * 0.01;
    if (getBalance() >= amt + fee && amt > 0) {
        withdraw(amt + fee);
        sendto(anotherAcnt);
        return true;
    }
    else
        return false;
}

public class Testcases{
protected void setUp()
{BasicAccount a=new Account(100.0, "user1 ");}
protected void tearDown(){}

public void test1(){
    a.transfer(50.0, "user2 ");
    a.withdraw(40.0);
    //Assertions
public void test2(){
    a.withdraw(50.0);
    a.sendto( "user2 ");
    //Assertions
public void test3(){
    a.withdraw(10.0);
    a.getBalance();
    //Assertions
public void test4(){
    a.deposit(30.0);
    //Assertions
public void test5(){
    a.getBalance();
    //Assertions
public void test6(){
    a.setBalance(30.0);
    //Assertions
}

```

(a) (b)

Fig. 2. Example program.

3 JUPTA

In this section, we first introduce the overall JUPTA approach (Section 3.1), and then describe several specific techniques based on that approach (Section 3.2).

3.1 Overview of JUPTA

The overall JUPTA approach can be summarized as follows:

1. Initially, the given test suite T contains all JUnit test cases to be prioritized and the prioritized test suite PS is empty.
2. On each iteration, JUPTA removes one test case with the highest testing ability (TA) from T and adds it to PS .
3. The preceding step is repeated until T becomes empty; then JUPTA stops the prioritization process and returns PS .

In this context, TA is a heuristic intended to help predict the ability of a test case to reveal unexposed faults. As it is unknown whether a test case will reveal faults before it has been run, TA is measured in terms of a test case's potential to cover system components (in terms of total components covered or components newly covered) as a proxy for an understanding of a test case's true fault-revealing potential.

As noted in Section 1, JUPTA, like other prioritization approaches, can be applied with or without feedback. In the first case, "total JUPTA" initially computes TA values for all test cases and then prioritizes all of them in terms of descending initial TA values. In the second case, "additional JUPTA" selects the best test case and then modifies the TA of each remaining test case to reflect coverage that might be obtained by test cases already prioritized.

To illustrate the overall approach, Fig. 2 provides an example Java class "Account" and its JUnit test cases. The class `Account` deals with operations on a bank account,

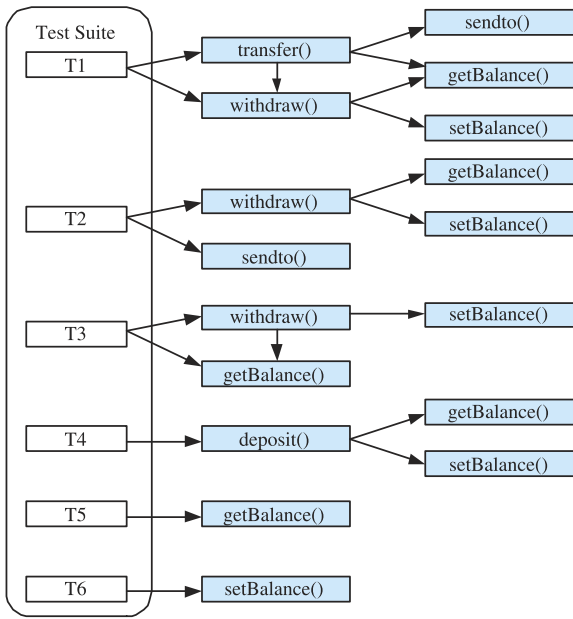


Fig. 3. Static call graph.

including deposit, withdraw, and money transfer. The JUnit class *Testcases* contains six test cases (i.e., test methods): *test1*, *test2*, *test3*, *test4*, *test5*, and *test6*.

Test case prioritization aims to schedule these six test cases according to their coverage of the *Account* class and the *BasicAccount* class. By constructing static call graphs of these test cases as shown in Fig. 3, we identify which methods of *Account* and *BasicAccount* may be executed when we run each test case in class *Testcases*. When we execute a test case, its callees will be invoked unless there are conditions governing the execution of callees. For example, in *test1* (denoted “T1” in Fig. 3), whether callee *sendto()* of *transfer()* is executed depends on the condition “*getBalance()* $\geq$ *amt*+*fee* && *amt* $>$ 0.”

Table 1 summarizes the callees of each test case (at the test-method level) following Fig. 3. Since we do not plan to run class *Testcases* before test case prioritization, we will not know which callees in Table 1 will definitely be executed, so we assume that all callees will be executed if we run the corresponding test case. Although this approach may overestimate the coverage of callees for each test case, the approach avoids the drawbacks associated with using dynamic coverage information.

Figs. 4 and 5 present the algorithms for total JUPTA and additional JUPTA, respectively. The algorithm for total JUPTA takes test suite *TS*, calculates a *TA* value for each test case $t \in TS$, and then places test cases in prioritized test suite *PS* in order of their *TA* values.

TABLE 1
Callees of Test Cases

Test Case	Callees
test1	transfer(double, String), withdraw(double), sendto(String), getBalance(), setBalance(double)
test2	withdraw(double), sendto(String), getBalance(), setBalance(double)
test3	withdraw(double), getBalance(), setBalance(double)
test4	deposit(double), getBalance(), setBalance(double)
test5	getBalance()
test6	setBalance(double)

Input: Test Case Set (denoted as *TS*)
Output: Prioritized Suite (denoted as *PS*)

General Process:

```

Begin
  for each test case t in TS
    calculate initial TA for t
  end for
  while TS is not empty do
    select t with the highest TA in TS
    TS ← TS - {t}
    add t to PS
  end while
  return PS
End
  
```

Fig. 4. Algorithm for total JUPTA.

Input: Test Case Set (denoted as *TS*)

Output: Prioritized Suite (denoted as *PS*)

Declaration: SlideSet: a data structure used to store test cases

General Process:

```

Begin
  SlideSet ← empty
  for each test case t in TS
    calculate initial TA for t
  end for
  while TS is not empty do
    for each test case t in TS
      adjust TA of t taking account
      the influence of test cases in SlideSet
    end for
    select t with the highest TA in TS
    if TA(t)=0.0 then
      SlideSet ← empty
      continue
    end if
    TS ← TS - {t}
    SlideSet ← SlideSet U {t}
    add t to PS
  end while
  return PS
End
  
```

Fig. 5. Algorithm for additional JUPTA.

The main difference between the total and additional JUPTA approaches lies in the approach used to achieve feedback and is denoted by the shaded rectangles in Fig. 5. In the test case prioritization process, the (adjusted) *TA* values of all test cases may reach 0 but *TS* may not be

empty yet. That is, although the test cases prioritized so far have achieved the maximum coverage, there are remaining test cases in the test suite to be prioritized and none can increase the current coverage. To handle this issue, additional JUPITA performs several prioritization cycles until all test cases have been prioritized. In each cycle, additional JUPITA takes the test cases to be prioritized as input, ignoring the test cases that have been prioritized in previous cycles, calculates the *TA* values of the input test cases, and selects the test case with the largest *TA*. As soon as a test case is selected, additional JUPITA recalculates the *TA* values of the remaining test cases in the input and selects the test case with the largest *TA*. In terms of the algorithm in Fig. 5, additional JUPITA uses *SlideSet* to store the set of selected test cases and recalculates the *TA* values of the test cases in *TS* as soon as a test case is selected and removed from *TS*. When this recalculation causes the *TA* values for all test cases remaining in *TS* to reach 0, the test cases prioritized so far have achieved maximal coverage, and no remaining test cases can increase coverage. Additional JUPITA then empties *SlideSet* and “continues” back to the while loop, where it resumes working with the rest of the not-yet-prioritized test cases in *TS*. Note that the first for loop within this while loop is responsible for adjusting the *TA* values for test cases in *TS*; when *SlideSet* is empty, this adjustment consists of resetting *TA* values to their initial nonempty values.

With both JUPITA algorithms, when JUPITA attempts to “select *t* with the highest *TA* in *TS*,” if multiple test cases have the same highest *TA* values, JUPITA selects a test case from among them using a secondary selection rule. Various secondary selection rules exist (see, e.g., [8], [12]), but the simplest secondary rule is to do such selection randomly and that is what we do in the implementations used in this work.

As we have noted, to prioritize test cases without relying on dynamic code coverage information, JUPITA analyzes the static call graphs of JUnit test cases and uses the callees of each JUnit test case as a basis for measuring the *TA* value of each JUnit test case. As the example discussed above shows, the *TA* value of a JUnit test case may relate to the methods called by the JUnit test case directly or indirectly in its static call graph. Before showing how JUPITA defines the formula for *TA*, we first provide the definition for determining a “relevant” relation between methods of the program under test and test cases as follows.

Definition 1. Let *m* be a method in a program under test; we say that *m* is relevant to a test case *t* iff *m* is included in the static call graph of *t*. In such a circumstance, we also say that *t* is relevant to *m*.

For our purposes, the *TA* value of a test case is defined based on the concept of the “relevant” relation. However, there are various possible formulations of *TA* that use the relevance notion differently and yield different JUPITA techniques, so we describe these specific formulations in Section 3.2.

The static call graphs required by JUPITA can be extracted from programs by various tools (e.g., [31], [32], [33]). Such extraction tools can differ quite widely, however, due to specific design decisions [27], [28] and algorithms (e.g., class hierarchy analysis (CHA) [26] and rapid type

analysis (RTA) [26], [29]). For example, according to Bacon’s [30] analysis, the expected-case running time of the algorithm for constructing a class hierarchy graph is $O(|CM|)$ (where *C* represents the set of classes and *M* represents the number of visible methods for a class in *C*), and the expected-case running time of RTA in practice is proportional to the size of *I* (where *I* represents the set of call instances). Moreover, Bacon [30] found that the complexity of RTA is no smaller than that of CHA and his experiments confirmed that the analysis time of RTA is usually larger than that of CHA.

Currently, JUPITA relies on static call graphs extracted by using CHA, which we implemented based on the ASM byte-code manipulation framework.⁴ This CHA algorithm is relatively inexpensive, but not as accurate as other algorithms. Because the CHA analysis cannot handle polymorphism and overloading, the static call graphs extracted by the CHA analysis are imprecise, overestimating possible call relationships. For example, in the program shown in Fig. 2, since *Account* overloads method *getBalance* of class *BasicAccount*, the CHA analysis does not know whether *T5* calls *Account.getBalance* or *BasicAccount.getBalance* in real executions and JUPITA thus assumes that the methods of static receiver class (i.e., *BasicAccount.getBalance*) are callees of *T5*.

Where time complexity is concerned, coverage-based prioritization techniques incur time and space costs collecting and storing code coverage data, and these time and space costs are not incurred by JUPITA. Beyond these costs, the time complexity of JUPITA’s algorithms is equivalent to that of traditional coverage-based test case prioritization techniques. Specifically, suppose that the number of test cases and the number of methods associated with the program *P* on which prioritization is being performed are *M* and *N*, respectively; the time complexity for the algorithm of total JUPITA is $O(MN) + O(M\log M)$, which is equivalent to the time complexity for traditional total test case prioritization techniques, and the time complexity for additional JUPITA is $O(M^2N) + O(M^2\log M)$, which is equivalent to the time complexity for traditional additional test case prioritization techniques [8].

3.2 JUPITA Techniques

There are three factors that can be considered in the construction of specific JUPITA techniques:

- As noted earlier, JUnit has two levels of test cases, test-method and test-class, and JUPITA can be implemented to schedule test cases at either level. For example, the test suite in Fig. 2 contains only one test case at the test-class level, which is class *Testcases*, but it contains six test cases at the test-method level: *test1*, *test2*, *test3*, *test4*, *test5*, and *test6*.
- Second, JUPITA uses *TA* values to guide the test case prioritization process. Although *TA* is proposed based on the “relevant” relation derived from static call graphs, *TA* can also be calculated based on two levels of granularity: fine granularity and coarse granularity. *TA* at the fine-granularity level is

4. <http://asm.ow2.org>.

TABLE 2
Techniques Studied

Techniques		Factors		
Approach Type	Abbreviation	Calculation Strategy	Coverage Information	Test Case Level
Random	R-M	–	–	Test-Method
	R-C	–	–	Test-Class
Untreated	U-M	–	–	Test-Method
	U-C	–	–	Test-Class
Optimal	O-M	–	–	Test-Method
	O-C	–	–	Test-Class
JUPTA	TJ-MM	Total	Method	Test-Method
	TJ-MC	Total	Method	Test-Class
	AJ-MM	Additional	Method	Test-Method
	AJ-MC	Additional	Method	Test-Class
	TJ-SM	Total	Statement	Test-Method
	TJ-SC	Total	Statement	Test-Class
	AJ-SM	Additional	Statement	Test-Method
	AJ-SC	Additional	Statement	Test-Class
Dynamic	TD-MM	Total	Method	Test-Method
	TD-MC	Total	Method	Test-Class
	AD-MM	Additional	Method	Test-Method
	AD-MC	Additional	Method	Test-Class
	TD-SM	Total	Statement	Test-Method
	TD-SC	Total	Statement	Test-Class
	AD-SM	Additional	Statement	Test-Method
	AD-SC	Additional	Statement	Test-Class

calculated based on the statements contained by the methods called by a test case, whereas TA at the coarse-granularity level is calculated based on the methods called by a test case.

- As we have noted, JUPTA can be applied with or without feedback (using the “total” or “additional” strategies).

Considering these three factors, we define eight different JUPTA techniques, TJ-MM, TJ-MC, AJ-MM, AJ-MC, TJ-SM, TJ-SC, AJ-SM, and AJ-SC, which are summarized in the middle eight rows of Table 2 and described in the following sections.

3.2.1 Total JUPTA Based on Method Coverage

TJ-MM is a “total” JUPTA technique, which prioritizes JUnit test cases at the test-method level based on estimated method coverage information. Specifically, TJ-MM calculates the TA of a test case t in T following (1), and schedules the order of test cases in T in terms of the descending order of their TA_{TJ-MM} :

$$TA_{TJ-MM}(t) = |\{m | m \in \gamma(t)\}|. \quad (1)$$

In the equation, $\gamma(t)$ denotes the set of methods relevant to t . As TA_{TJ-MM} of test case t is the number of methods that are relevant to t , a test case with a larger number of descendants has greater testing ability than a test case with a smaller number.⁵

TJ-MC is similar to TJ-MM, but TJ-MC prioritizes test cases at the test-class level instead of the test-method level. During the prioritization process, TJ-MC treats each test-class (e.g., *Testcases* in Fig. 2) as one test case and prioritizes these test cases based on their relevant methods.

5. Furthermore, TA_{TJ-MM} can be extended to be defined by the sum of probability that method m contains faults in its body, which may depend on many factors, including developer ability, method complexity, change history, and so forth. Thus, (1) is an instance of this extension.

3.2.2 Additional JUPTA Based on Method Coverage

AJ-MM is an “additional” JUPTA technique, which prioritizes JUnit test cases at the test-method level based on estimated method coverage information. The only difference between TJ-MM and AJ-MM involves different TA calculation since AJ-MM is an “additional” technique. Initially, AJ-MM selects any test case t with the highest TA_{TJ-MM} . After removing t from T and adding it to PS , AJ-MM adds t to *SlideSet* and adjusts the TA values of all the remaining test cases in T following (2):

$$TA_{AJ-MM}(t) = \left| \left\{ m | m \in \gamma(t) - \bigcup_{t' \in SS} \gamma(t') \right\} \right|. \quad (2)$$

In the equation, SS denotes *SlideSet*. In (2), the methods included in the call graphs of existing selected test cases in *SlideSet* will be ignored during TA adjustment because unselected test cases covering these methods may expose faults that have not been exposed by the selected test cases.

AJ-MC is similar to AJ-MM, but AJ-MC prioritizes test cases at the test-class level.

3.2.3 Total JUPTA Based on Statement Coverage

TJ-SM is a “total” JUPTA technique that prioritizes JUnit test cases at the test-method level based on estimated statement coverage information. The (estimated) statement coverage used by JUPTA is a set of statements contained by the methods that are relevant to a test case. Specifically, TJ-SM calculates TA based on (3):

$$TA_{TJ-SM}(t) = \left| \left\{ s | s \in \lambda(m) \bigwedge m \in \gamma(t) \right\} \right|. \quad (3)$$

In the equation, $\lambda(m)$ is the set of statements contained in method m . That is, TJ-SM uses the number of statements contained in the methods relevant to t to measure the testing ability of test case t and then schedules the test cases in T based on the descending order of their TA values.

TABLE 3
Object Characteristics

Object	Source code under test			JUnit test suite			
	#Classes	#Methods	LOC	#TestMethod	#TestClass	Method Coverage(%)	Statement Coverage(%)
jtopas-v1	19	284	1,894	126	10	82.0	85.0
jtopas-v2	21	302	2,031	128	11	82.2	82.4
jtopas-v3	50	748	5,361	209	18	73.1	76.1
xmlsec-v1	179	1,627	18,323	92	15	28.9	32.4
xmlsec-v2	180	1,629	18,985	94	15	28.4	32.0
xmlsec-v3	145	1,398	16,878	84	13	29.7	34.8
jmeter-v1	334	2,919	33,670	78	26	16.1	17.1
jmeter-v2	319	2,838	33,097	80	29	23.6	25.9
jmeter-v3	373	3,445	37,271	78	33	25.9	26.5
jmeter-v4	380	3,536	38,357	78	33	25.9	26.2
jmeter-v5	389	3,613	41,052	97	37	26.2	26.3
ant-v1	128	2,511	25,846	137	34	18.0	17.9
ant-v2	342	3,836	39,733	219	51	20.7	20.8
ant-v3	342	3,845	39,810	219	51	20.4	20.7
ant-v4	532	5,684	61,877	521	102	30.6	31.9
ant-v5	536	5,802	63,510	557	105	30.8	32.2
ant-v6	536	5,808	63,578	559	105	30.9	32.2
ant-v7	649	7,520	80,381	877	149	33.4	34.9
ant-v8	650	7,524	80,444	878	149	33.7	35.1

TJ-SC is similar to TJ-SM, but TJ-SC prioritizes test cases at the test-class level.

3.2.4 Additional JUPTA Based on Statement Coverage

AJ-SM is the “additional” implementation of TJ-SM, which follows the algorithm given in Fig. 5. Initially, AJ-SM uses the number of statements contained in the methods relevant to any test case t to measure its TA value. During test case prioritization, AJ-SM adjusts the TA value of any unselected test case t by the number of statements contained by the methods that are relevant to t and uncovered by the existing selected test cases.

AJ-SC is similar to AJ-SM, but AJ-SC prioritizes test cases at the test-class level.

4 EMPIRICAL STUDY

To investigate the effectiveness of JUPTA techniques relative to existing prioritization techniques, we designed and conducted a controlled experiment.

4.1 Research Questions

Our experiment addresses the following research questions:

1. **How do the eight JUPTA techniques compare in terms of effectiveness?** As we have proposed eight JUPTA techniques, we wish to know which of them is more effective in test case prioritization.
2. **Is JUPTA more effective than random or untreated test case orders?** Random and untreated orders are often used as controls in experiments on test case prioritization; we use them similarly here.
3. **How does JUPTA compare to an optimal test case order?** Since it is unknown which faults each test case will reveal before we run test cases, it is impossible to know a priori whether a scheduled test suite is the best. However, in a study where faults are known, it is possible to calculate optimal orders and measure their effectiveness.

4. **How does JUPTA compare to dynamic coverage-based test case prioritization techniques?** Because dynamic techniques use actual coverage information to estimate the fault-detection ability of a test case, we expect them to be more effective than JUPTA.

In the following sections, we provide details on our objects of study (Section 4.2), our independent and dependent variables and measures (Sections 4.3 and 4.4), and our experiment process and threats to validity (Sections 4.5 and 4.6). Section 4.7 presents our results, and Section 4.8 provides additional discussion.

4.2 Objects of Study

As objects of study, we selected four nontrivial Java programs that have been used widely in prior studies of prioritization, and are available from the Software-artifact Infrastructure Repository [34] (SIR⁶). Jtopas is a Java library used for parsing text data. Xmlsec is a component library implementing XML⁷ signature and encryption standards. Jmeter is a Java desktop application used to load test functional behavior and measure performance. Ant is a Java-based build tool, similar to the Unix tool “make.” Each of these programs has several versions available in SIR.

Our study used all the versions of these programs with seeded faults available in SIR and thus involved 19 objects, whose basic information is shown in Table 3. In the table, the second, third, and fourth columns give the number of classes, the number of methods, and the number of lines of code (excluding comments), respectively. Each of these objects is equipped with JUnit test cases. As JUPTA can prioritize test cases at the test-method and test-class levels, the fifth and sixth columns of this table present the numbers of test methods and test classes, respectively. The last two columns of this table list the percentage of methods and statements executed by the test suites.

6. <http://sir.unl.edu/portal/index.html>.

7. XML is the abbreviation of extensible markup language, which is a set of rules for encoding documents in machine readable form.

TABLE 4
Faults in the Objects

Object	#Seeded faults	#Mutants	#Groups of mutants	#Mutants per group	Mutant killing rate
jtopas-v1	10	349	20	5	0.3576
jtopas-v2	12	443	20	5	0.3989
jtopas-v3	16	4,830	20	5	0.7633
xmlsec-v1	20	2,824	20	5	0.1608
xmlsec-v2	19	2,814	20	5	0.1872
xmlsec-v3	13	2,575	20	5	0.0652
jmeter-v1	19	3,717	7	5	0.0094
jmeter-v2	20	3,723	20	5	0.0944
jmeter-v3	19	4,092	20	5	0.0913
jmeter-v4	13	3,155	20	5	0.0859
jmeter-v5	12	3,489	20	5	0.0864
ant-v1	11	14,712	20	5	0.2255
ant-v2	21	14,484	20	5	0.1997
ant-v3	7	14,582	20	5	0.1996
ant-v4	26	30,741	20	5	0.4612
ant-v5	15	30,980	20	5	0.4151
ant-v6	1	30,969	20	5	0.4578
ant-v7	29	37,919	20	5	0.5556
ant-v8	4	37,943	20	5	0.4437

These object programs already have seeded faults [34], which were manually injected by previous researchers and collected by SIR. The number of seeded faults for each object is provided in the second column of Table 4. In addition to these faulty versions, for our study we also constructed additional faulty versions using program mutation, following procedures similar to those described in [35]. For each object version, we used μ Java⁸ [36] to generate a pool of mutants,⁹ each of which is the result of applying a mutation operator (e.g., replace a Boolean expression with true or false) to the source code. The total number of generated mutants is shown in the third column of Table 4. As some mutants cannot be exposed by the given test cases, in the last column of Table 4 we give the ratio between the number of mutants exposed by the test suites for each object program and the total number of generated mutants. For the purposes of our study, we generated several mutant groups, each of which contains five randomly selected mutants that have been killed by the given test suite. Then, we use each mutant group to construct and simulate a multiple-fault version as done in [37]. In Table 4, the fourth and fifth columns give the number of mutant groups and the number of mutants contained in each group, respectively. Jmeter version 1 has only seven groups of mutants because our mutation pool does not contain enough killable mutants to construct more.

4.3 Independent Variable

Our independent variable is the prioritization technique, and the techniques we consider are summarized in Table 2. Among the eight JUPA techniques, TJ-MM and AJ-MM have been proposed and studied in our previous work [17]. In addition to these eight techniques, we implemented another 14 prioritization techniques listed in Table 2, as follows:

- **Random test case prioritization** orders test cases randomly. Specifically, this approach randomly¹⁰ removes a test case from the given test suite T and adds it into the prioritized test suite PS until T is empty. To reduce the influence of random selection and achieve stable results for the random test case prioritization approach, for our study we applied our random test case prioritization techniques 20 times and calculated average results. In our discussion, the random technique which prioritizes JUnit test cases at the test-method level is denoted by “R-M,” whereas the random technique to prioritizing JUnit test cases at the test-class level is denoted by “R-C.”
- **Untreated test case prioritization** keeps the original order of test cases as provided by SIR. In our discussion, the untreated technique which prioritizes JUnit test cases at the test-method level is denoted by “U-M,” whereas the untreated technique which prioritizes test cases at the test-class level is denoted by “U-C.”
- **Optimal test case prioritization** is used to provide an ideal permutation on a test suite. Specifically, this optimal approach prioritizes test cases based on the number of faults that they actually reveal given that it is known, after all test cases have been run, which faults are revealed by each test case. While not a feasible approach in practice where fault detection of tests is unknown a priori, the approach is applicable in an empirical study where fault-detection results can be measured. The optimal approach can also be implemented to prioritize test cases either at the test-class level or at the test-method level. We denote the former technique by “O-M” and the latter by “O-C.”
- **Dynamic coverage-based test case prioritization techniques.** Prior work has focused on prioritization using dynamic coverage information, and the cost effectiveness of JUPA needs to be compared to this.

8. μ Java is a mutation system for Java programs and is available at <http://cs.gmu.edu/~offutt/mujava/>.

9. μ Java generates two types of mutation operators for traditional mutation and class mutation, respectively. We used only traditional mutation in this experiment.

10. Specifically, we generated random numbers using the system time as seeds.

To do this, we implemented dynamic techniques following the algorithm of JUPTA and varying the same three factors (types of test cases, types of coverage information used, and strategies). However, the dynamic techniques use code coverage measured by instrumentation. For discussion, we denote these by “TD-MM,” “TD-MC,” “AD-MM,” “AD-MC,” “TD-SM,” “TD-SC,” “AD-SM,” and “AD-SC.”

Note that there is one important difference in our application of the traditional dynamic coverage-based techniques from that used in the literature. In practice, a prioritization technique that uses coverage data relies on data gathered on the prior version of a system to prioritize tests for a new version, and this is a source of imprecision because coverage data on a prior version are not necessarily what they would be on a new version. In our study, we apply the comparator techniques to coverage data gathered on the new release. This allows us to obtain the result that those coverage-based techniques could achieve in the absence of the decay of coverage information, eliminating any loss of precision that might result by prioritizing based on the prior release’s coverage. This allows us to avoid a potentially confounding factor related to the decay of coverage information.

4.4 Dependent Variable

As a dependent variable, we use the **Average Percentage Faults Detected (APFD)** metric [1]. This metric measures prioritization effectiveness in terms of the rate of fault detection of a test suite, and is defined by the following formula:

$$APFD = 1 - \frac{\sum_{i=1}^{num_f} TF_i}{num_t * num_f} + \frac{1}{2 * num_t}. \quad (4)$$

In the formula, num_t denotes the total number of test cases, num_f denotes the total number of detected faults, and TF_i ($1 \leq i \leq num_f$) denotes the smallest number of test cases in sequence that need to be run in order to expose fault i . APFD values range is $[0, 1]$. For any given test suite, its num_t and num_f are fixed so that higher APFD values signal that the average value of TF_i (variable i ranges from 1 to num_f) is lower and thus imply a higher fault-detection rate.

4.5 Experiment Process

For each object program and fault, we executed each of the program’s test cases on that program with that fault present to determine whether the test case can detect it. We recorded fault-detection information in a table so that we could later compute the APFD value of each prioritized test suite easily without running each prioritized test suite.

For each of the object programs considered, for each version, we next applied each of the 22 test case prioritization techniques to the version and its associated JUnit test suite. We then calculated the APFD values for the prioritized suites relative to the seeded faults and the groups of mutants.

Because the dynamic coverage-based prioritization techniques that we consider prioritize test cases based on actual execution information, we obtained that information using an instrumentation tool implemented within the ASM byte

code manipulation framework.¹¹ We performed our experiment and gathered all cost data on an AMD Athlon II P340 Dual-Core Processor 2.20 GHz.

4.6 Threats to Validity

The main threat to internal validity for this experiment involves possible faults in our implementation of the techniques that we study and scripts that control the experiment process. To reduce this threat, we carefully inspected all of the code produced and tested it on known cases.

The main threat to external validity for this study lies in the objects used. We conducted the study on 19 versions of four open source Java programs ranging from 2 to 80 KLOC. However, these programs are not necessarily representative of other programs. Another threat to external validity is the quality of test cases. To reduce this threat, we use the test cases developed in the real evolution of objects. The faults in the object programs are also a threat to external validity. To reduce this threat, we generated faults using a mutation testing tool and used these in addition to the fault seeded versions of objects from SIR. Still, these faults may not be representative of faults found “in the wild.”

The main threat to construct validity for this study is the metric used to assess the effectiveness of prioritization techniques. The APFD metric that we use has been extensively used in prior work, but it does not capture all of the costs and benefits related to test case prioritization [38]. For example, it does not account for the fact that test cases can have different execution times and that faults can have different severities nor does it account for factors such as costs of the analysis used to perform prioritization or the human costs associated with creating, executing, and assessing the outcomes of tests.

4.7 Results and Analysis

In this section, we first use descriptive statistics to summarize the overall experiment results considering all programs and fault types together, and then we further analyze the results separately per program, for the cases in which faults were seeded by SIR and in which faults were generated by μ Java.

4.7.1 Overall Results

To provide an overall view of the results, Fig. 6 uses boxplots¹² to present the APFD values for all programs and versions considered together, per prioritization technique. The horizontal axis lists the names of prioritization techniques, and the vertical axis presents APFD values. The figure is divided into three portions by two lines. The leftmost portion presents data gathered for the control techniques (random including R-M and R-C, untreated including U-M and U-C, and optimal techniques including O-M and O-C). The middle portion presents data gathered

11. <http://asm.ow2.org/>.

12. A boxplot is a standard statistical device for representing data distributions; boxplots have an advantage over simple averages in that they better depict the variance among elements of the dataset. In the boxplots presented in this paper, each dataset’s distribution is represented by a box. The box’s width spans the central 50 percent of the data, and its ends mark the upper and lower quartiles. The vertical line partitioning the box represents the median element in the dataset; the mean value for the set is also depicted using a small rectangle. The horizontal lines attached to the ends of the box indicate the tails of the distribution.

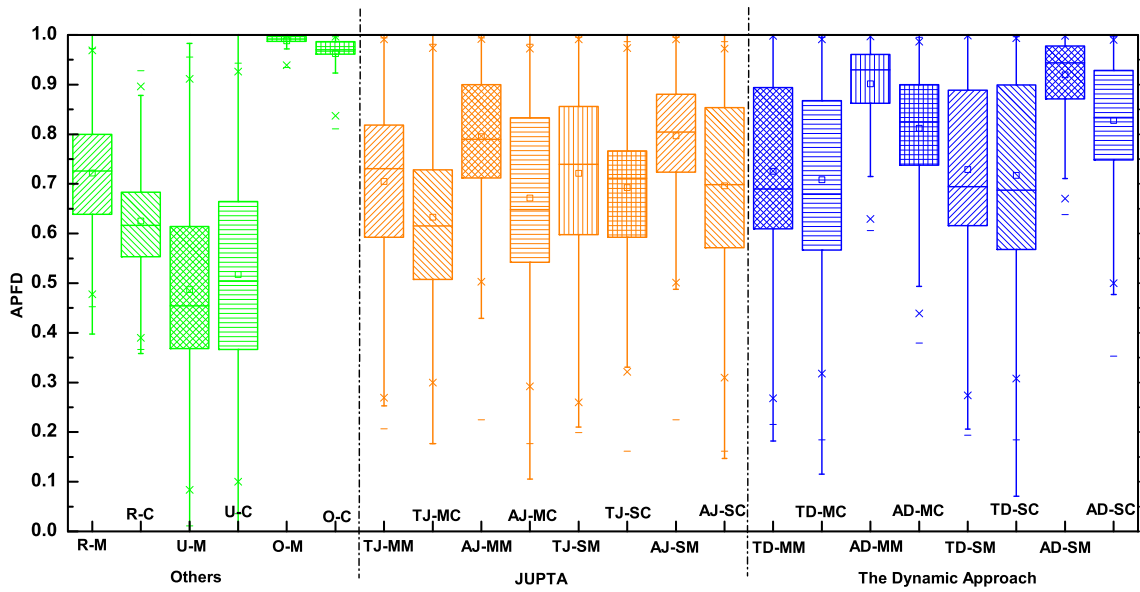


Fig. 6. APFD boxplots for all programs.

for the JUPTA techniques, and the rightmost portion presents data for the dynamic coverage-based techniques.

First, where comparisons with control techniques are concerned, the boxplots suggest that the results of the optimal approach are higher—typically much more so—than those observed for other techniques, and exhibit little variance. Since the optimal approach provides an ideal permutation on a test suite, the results of the optimal approach serve as an upper bound on prioritization results. The JUPTA and dynamic coverage-based heuristics do occasionally achieve this upper bound, but only on a small proportion of their executions. Untreated orders appear to exhibit the lowest values and relatively large variance, and random orders appear to be better than these with less variance. All of the heuristics appear to outperform untreated orders, while many (but not all) appear to outperform random orders.

Second, where comparisons between dynamic coverage-based heuristics and JUPTA heuristics are concerned, the former appear to usually achieve higher mean values than their corresponding JUPTA heuristics. The boxplots for AJ-MM and AJ-SM, however, do exhibit means that appear greater than those of all but two dynamic coverage-based techniques.

Third, where comparisons between types of test cases in JUPTA are concerned, the boxplots suggest that the prioritization results of JUnit test cases at the test-method level are usually higher than those of JUnit test cases at the test-class level. That is, JUPTA is more effective at prioritizing test cases at the test-method level than at the test-class level.

4.7.2 Results and Analysis Using Seeded Faults

To investigate whether there are significant differences among the 22 test case prioritization techniques, we performed a one-way ANOVA¹³ on the APFD values obtained from each object program considering seeded

faults. The analysis results for the four object programs and all the object programs (denoted by “All”) are shown in Table 5, where SS denotes Sum of Squares, DF denotes Degrees of Freedom, MS denotes Mean Square, F denotes the statistical F-value, and p denotes the calculated p-value (which is represented by 0.000 when it is less than 0.001).

As the table shows, in all cases except for `xmlsec`, the p-values are smaller than the significance level utilized ($\alpha = 0.05$). Therefore, there are significant differences among the 22 test case prioritization techniques on `jtopas`, `jmeter`, and `ant`. Moreover, the one-way ANOVA analysis on all the object programs taken together confirms that the 22 test case prioritization techniques have significant differences overall.

To further examine the differences between individual techniques, we performed a Tukey HSD posthoc test on the experiment data of each object program other than `xmlsec`

TABLE 5
One-Way ANOVA Results, Seeded Faults

jtopas	SS	DF	MS	F	p
Between Groups	1.465	21	0.070	1.856	0.042
Within Groups	1.653	44	0.038		
Total	3.118	65			
xmlsec	SS	DF	MS	F	p
Between Groups	0.308	21	0.015	1.185	0.309
Within Groups	0.545	44	0.012		
Total	0.853	65			
jmeter	SS	DF	MS	F	p
Between Groups	2.835	21	0.135	11.745	0.000
Within Groups	1.011	88	0.011		
Total	3.846	109			
ant	SS	DF	MS	F	p
Between Groups	4.351	21	0.207	7.386	0.000
Within Groups	4.320	154	0.028		
Total	8.670	175			
All	SS	DF	MS	F	p
Between Groups	7.330	21	0.349	12.502	0.000
Within Groups	11.056	396	0.028		
Total	18.385	417			

13. The statistical analyses (including the one-way ANOVA and Tukey HSD posthoc tests) described in this paper were performed using SPSS 15.0, analytical software accessible at <http://www.spss.com>.

TABLE 6
Tukey HSD Posthoc Test, *jtopas*, Seeded Faults

Grouping	Means	Techniques	
		Abbreviation	Name
A	0.97604000	O-M	optimal-method
A	0.92405333	O-C	optimal-class
A	0.86104000	AD-SM	dynamic-additional-statementcoverage-method
A	0.83084000	AD-MM	dynamic-additional-methodcoverage-method
A	0.75862333	AJ-SM	JUPTA-additional-statementcoverage-method
A	0.75055333	AD-MC	dynamic-additional-methodcoverage-class
A	0.73985000	AJ-MM	JUPTA-additional-methodcoverage-method
A	0.69499667	AD-SC	dynamic-additional-statementcoverage-class
A	0.66628667	TJ-SM	JUPTA-total-statementcoverage-method
A	0.66433667	R-M	random-method
A	0.64781667	TJ-MM	JUPTA-total-methodcoverage-method
A	0.64164667	AJ-SC	JUPTA-additional-statementcoverage-class
A	0.62400667	TJ-SC	JUPTA-total-statementcoverage-class
A	0.58885667	R-C	random-class
A	0.57242000	TJ-MC	JUPTA-total-methodcoverage-class
A	0.54023333	AJ-MC	JUPTA-additional-methodcoverage-class
A	0.50893000	TD-MC	dynamic-total-methodcoverage-class
A	0.49305333	TD-SC	dynamic-total-statementcoverage-class
A	0.48440667	U-M	untreated-method
A	0.48351667	TD-SM	dynamic-total-statementcoverage-method
A	0.48072000	TD-MM	dynamic-total-methodcoverage-method
A	0.43441333	U-C	untreated-class

considering seeded faults. The results of the tests for the three object programs are shown in Tables 6, 7, and 8, and the results of the test for all the object programs are shown in Table 9. The second column of the table lists the mean APFD for each technique, and the third and fourth columns list technique abbreviations and full names. The first column, "Grouping," assigns grouping letters to techniques such that techniques that are not statistically significantly different from one another share grouping letters.

As Table 6 shows, where *jtopas* is concerned, the Tukey test fails to indicate significant differences between individual techniques, despite the indication by the

ANOVA that differences exist. This is likely due to differences in the power of the statistical tests. On *ant* and *jmeter*, as well as on programs overall, significant differences are seen between classes of techniques, and these do reflect the overall picture presented in the boxplots. In these three cases, the optimal technique is always in the most effective grouping of techniques, while the untreated is always in the least effective and the random technique is in either the least or second-to-least effective group.

Where comparisons of the heuristic techniques are concerned, four of the techniques (AD-MM, AJ-MM, AD-

TABLE 7
Tukey HSD Posthoc Test, *jmeter*, Seeded Faults

Grouping	Means	Techniques	
		Abbreviation	Name
A	0.98676400	O-M	optimal-method
A	0.96671800	O-C	optimal-class
A B	0.87287600	AD-MM	dynamic-additional-methodcoverage-method
A B C	0.85789200	AJ-MM	JUPTA-additional-methodcoverage-method
A B C D	0.84345000	AD-SM	dynamic-additional-statementcoverage-method
A B C D E	0.81848800	AJ-SM	JUPTA-additional-statementcoverage-method
A B C D E F	0.77624800	AD-MC	dynamic-additional-methodcoverage-class
A B C D E F	0.76442400	AD-SC	dynamic-additional-statementcoverage-class
A B C D E F	0.76394400	AJ-MC	JUPTA-additional-methodcoverage-class
B C D E F	0.70930600	AJ-SC	JUPTA-additional-statementcoverage-class
B C D E F	0.69699000	R-M	random-method
B C D E F	0.63885600	TD-MC	dynamic-total-methodcoverage-class
B C D E F	0.63720000	TD-SC	dynamic-total-statementcoverage-class
C D E F G	0.61884200	TJ-MC	JUPTA-total-methodcoverage-class
C D E F G	0.61232800	TJ-MM	JUPTA-total-methodcoverage-method
D E F G	0.60078200	TJ-SC	JUPTA-total-statementcoverage-class
E F G	0.58876200	TD-SM	dynamic-total-statementcoverage-method
E F G	0.57309400	TD-MM	dynamic-total-methodcoverage-method
F G	0.55273400	R-C	random-class
F G	0.55247200	TJ-SM	JUPTA-total-statementcoverage-method
G	0.37798600	U-M	untreated-method
G	0.37657000	U-C	untreated-class

TABLE 8
Tukey HSD Posthoc Test, ant, Seeded Faults

Grouping	Means	Techniques	
		Abbreviation	Name
A	0.99653625	O-M	optimal-method
A	0.98477625	O-C	optimal-class
A B	0.89658250	AJ-MM	JUPTA-additional-methodcoverage-method
A B C	0.88414375	AJ-SM	JUPTA-additional-statementcoverage-method
A B C	0.84506625	AD-SM	dynamic-additional-statementcoverage-method
A B C	0.82697000	TJ-MM	JUPTA-total-methodcoverage-method
A B C	0.81612625	TJ-SM	JUPTA-total-statementcoverage-method
A B C	0.81531750	AD-MM	dynamic-additional-methodcoverage-method
A B C	0.80000000	AJ-MC	JUPTA-additional-methodcoverage-class
A B C	0.78424875	AJ-SC	JUPTA-additional-statementcoverage-class
A B C D	0.74198500	TJ-MC	JUPTA-total-methodcoverage-class
A B C D	0.74065000	TJ-SC	JUPTA-total-statementcoverage-class
A B C D	0.70889250	R-M	random-method
B C D E	0.66496625	AD-SC	dynamic-additional-statementcoverage-class
B C D E	0.65699125	AD-MC	dynamic-additional-methodcoverage-class
B C D E	0.60164250	R-C	random-class
B C D E	0.59043625	TD-SC	dynamic-total-statementcoverage-class
C D E	0.58781875	TD-SM	dynamic-total-statementcoverage-method
C D E	0.57908500	TD-MM	dynamic-total-methodcoverage-method
C D E	0.57864875	TD-MC	dynamic-total-methodcoverage-class
D E	0.44928875	U-C	untreated-class
E	0.38213625	U-M	untreated-method

SM, and AJ-SM), including two dynamic coverage-based and two JUPTA techniques, are statistically significantly the same as the optimal techniques, as well as each other. Also, in these three cases, techniques employing feedback (“additional” techniques) tend to fall into statistically significantly higher performing groups than corresponding techniques that do not employ feedback (“total” techniques) for both the JUPTA and dynamic-coverage-based techniques. However, statement coverage and method coverage variations of techniques (of comparable technique types) seldom differ significantly.

4.7.3 Results and Analysis Using Mutation Faults

Table 10 shows the results of applying the one-way ANOVA to the APFD results for each object program as well as all the programs considering mutation faults. The results here too show that there are significant differences among the 22 techniques—in this case for each program and for all programs considered together.

To further analyze the data for differences between individual techniques, we performed the Tukey HSD posthoc test on the results for each program separately, and all programs. The results of the tests for each object program are given in Tables 11, 12, 13, and 14 and the

TABLE 9
Tukey HSD Posthoc Test, All Programs, Seeded Faults

Grouping	Means	Techniques	
		Abbreviation	Name
A	0.98981611	O-M	optimal-method
A	0.96580256	O-C	optimal-class
A B	0.86619032	AD-SM	dynamic-additional-statementcoverage-method
A B	0.85531696	AJ-MM	JUPTA-additional-methodcoverage-method
A B	0.85173617	AD-MM	dynamic-additional-methodcoverage-method
A B	0.84079478	AJ-SM	JUPTA-additional-statementcoverage-method
B C	0.75898117	AJ-MC	JUPTA-additional-methodcoverage-class
B C	0.75147612	AJ-SC	JUPTA-additional-statementcoverage-class
B C	0.74551318	TJ-MM	JUPTA-total-methodcoverage-method
B C	0.73523199	AD-MC	dynamic-additional-methodcoverage-class
B C	0.73372388	AD-SC	dynamic-additional-statementcoverage-class
B C	0.72811260	TJ-SM	JUPTA-total-statementcoverage-method
B C	0.72076142	R-M	random-method
B C	0.70044779	TJ-MC	JUPTA-total-methodcoverage-class
B C	0.70027097	TJ-SC	JUPTA-total-statementcoverage-class
C D	0.63998517	TD-SC	dynamic-total-statementcoverage-class
C D	0.63696157	TD-MC	dynamic-total-methodcoverage-class
C D	0.62570042	TD-SM	dynamic-total-statementcoverage-method
C D	0.61693909	TD-MM	dynamic-total-methodcoverage-method
C D	0.61526321	R-C	random-class
D	0.47508076	U-C	untreated-class
D	0.44809433	U-M	untreated-method

TABLE 10
One-Way ANOVA Results, Mutation Faults

jtopas	SS	DF	MS	F	p
Between Groups	15.739	21	0.749	28.566	0.000
Within Groups	34.056	1298	0.026		
Total	49.796	1319			
xmlsec	SS	DF	MS	F	p
Between Groups	32.656	21	1.555	123.570	0.000
Within Groups	16.335	1298	0.013		
Total	48.991	1319			
jmeter	SS	DF	MS	F	p
Between Groups	50.771	21	2.418	152.827	0.000
Within Groups	29.931	1892	0.016		
Total	80.701	1913			
ant	SS	DF	MS	F	p
Between Groups	53.536	21	2.549	158.718	0.000
Within Groups	56.185	3498	0.016		
Total	109.721	3519			
All	SS	DF	MS	F	p
Between Groups	6.266	21	0.298	22.297	0.000
Within Groups	5.299	396	0.013		
Total	11.566	417			

results of the test for all programs are given in Table 15. In this case, results yield more grouping levels than those obtained on the seeded faults, most likely due to the presence of larger numbers of groups of faults, as supported by the larger numbers of mutation faults available overall.

As Tables 11, 12, 13, 14, and 15 show, the optimal test case prioritization techniques, O-M and O-C, are again among the most effective techniques, and share this distinction with fewer heuristics. Untreated and random orders consistently fall into the worst or second worst effective groupings with the exception of R-M which falls into groupings that lie in the middle in terms of statistically significant differences.

Comparing the eight JUPTA techniques and the eight dynamic coverage-based techniques, we note that the two

most effective dynamic coverage-based techniques continue to be AD-SM and AD-MM. AD-SM is statistically equivalent to the optimal technique on three of four programs (jmeter excepted) and all programs overall, and AD-MM is statistically significantly equivalent to the optimal technique on two of four programs (jtopas and jmeter excepted). The AJ-SM and AJ-MM techniques do remain statistically significantly equivalent to their dynamic coverage-based comparands on two programs, jtopas and jmeter, while not faring as well on the other two programs.

We also continue to note that techniques using feedback are better than those that do not. In terms of means, this occurs in all cases; also, AD-SM is statistically significantly better than TD-SM, and AD-MM is statistically significantly better than TD-MM. This observation is consistent with our observation on results of some object programs (i.e., jmeter and all programs) with seeded faults.

4.8 Summary and Discussion

We now summarize the results of our empirical study in reference to the research questions raised at the beginning of Section 4:

1. Among the eight JUPTA techniques, techniques using feedback tend to be more effective than their corresponding techniques that do not use feedback, and techniques using feedback at the test-method level (AJ-MM and AJ-SM) tend to be the most effective. Techniques using feedback retain the advantage over techniques without feedback because the latter may postpone the detection of faults in rarely covered statements.
2. JUPTA is more effective than an approach that reuses untreated test suites. JUPTA is also more effective than the random approach at the test-method level, and usually more effective than the random approach at the test-class level. As JUPTA

TABLE 11
Tukey HSD Posthoc Test, jtopas, Mutation Faults

Grouping	Means	Techniques	
		Abbreviation	Name
A	0.96730233	O-M	optimal-method
A B	0.90101833	O-C	optimal-class
A B C	0.88506050	AD-SM	dynamic-additional-statementcoverage-method
A B C	0.86685433	AJ-SM	JUPTA-additional-statementcoverage-method
B C D	0.84534883	AD-MM	dynamic-additional-methodcoverage-method
B C D E	0.82673950	AJ-MM	JUPTA-additional-methodcoverage-method
B C D E F	0.80666617	AD-SC	dynamic-additional-statementcoverage-class
B C D E F G	0.80189983	TJ-SM	JUPTA-total-statementcoverage-method
C D E F G	0.79361033	AD-MC	dynamic-additional-methodcoverage-class
C D E F G	0.78403617	R-M	random-method
C D E F G	0.77983000	TJ-MM	JUPTA-total-methodcoverage-method
D E F G	0.74208233	TJ-SC	JUPTA-total-statementcoverage-class
E F G H	0.73041033	TD-MM	dynamic-total-methodcoverage-method
E F G H	0.72240750	AJ-SC	JUPTA-additional-statementcoverage-class
F G H	0.71940783	TD-SM	dynamic-total-statementcoverage-method
F G H	0.71847183	TJ-MC	JUPTA-total-methodcoverage-class
F G H	0.70847150	TD-MC	dynamic-total-methodcoverage-class
F G H	0.70062250	R-C	random-class
G H	0.69564750	TD-SC	dynamic-total-statementcoverage-class
H I	0.62861000	AJ-MC	JUPTA-additional-methodcoverage-class
I J	0.55613583	U-M	untreated-method
J	0.47773100	U-C	untreated-class

TABLE 12
Tukey HSD Posthoc Test, *xmlsec*, Mutation Faults

Grouping	Means	Techniques	
		Abbreviation	Name
A	0.98984350	O-M	optimal-method
A B	0.95871917	O-C	optimal-class
A B	0.95389317	AD-SM	dynamic-additional-statementcoverage-method
A B	0.94751017	AD-MM	dynamic-additional-methodcoverage-method
B C	0.89398233	AD-MC	dynamic-additional-methodcoverage-class
C D	0.86601500	AD-SC	dynamic-additional-statementcoverage-class
D E	0.81945317	AJ-SC	JUPTA-additional-statementcoverage-class
E F	0.76385833	TD-SM	dynamic-total-statementcoverage-method
E F	0.76238133	AJ-SM	JUPTA-additional-statementcoverage-method
E F	0.76125683	TD-MM	dynamic-total-methodcoverage-method
F G	0.73919567	AJ-MM	JUPTA-additional-methodcoverage-method
F G H	0.73438167	TJ-SM	JUPTA-total-statementcoverage-method
F G H	0.73345483	TJ-SC	JUPTA-total-statementcoverage-class
F G H	0.72331383	R-M	random-method
F G H	0.71087183	TD-SC	dynamic-total-statementcoverage-class
G H I	0.67758517	TJ-MM	JUPTA-total-methodcoverage-method
H I	0.66355733	TD-MC	dynamic-total-methodcoverage-class
I J	0.60524983	AJ-MC	JUPTA-additional-methodcoverage-class
J K	0.56971150	R-C	random-class
K	0.51934867	TJ-MC	JUPTA-total-methodcoverage-class
L	0.44201933	U-C	untreated-class
L	0.42027983	U-M	untreated-method

schedules test cases based on their *TA* values, the test suites generated by JUPTA are intuitively better than the test suites in random or untreated orders on fault-detection effectiveness.

- Overall, an optimal test ordering is most consistently preferential to other approaches including JUPTA. Thus, existing test case prioritization techniques including JUPTA techniques can still be improved.
- As dynamic coverage-based techniques use actual coverage information while JUPTA uses estimated coverage information, the former is intuitively better than the latter in terms of fault-detection effectiveness.

According to the results of the empirical study, dynamic coverage-based techniques can be more effective than JUPTA techniques at detecting seeded faults, but JUPTA techniques AJ-MM and AJ-SM are often just as effective as dynamic coverage-based techniques.

The foregoing results suggest that in cases where gathering test execution data is not desirable (e.g., for reasons such as those described in Section 1), JUPTA is a reasonable alternative to consider.

In this empirical study, we compared JUPTA with several existing test case prioritization techniques in terms of

TABLE 13
Tukey HSD Posthoc Test, *jmeter*, Mutation Faults

Grouping	Means	Techniques	
		Abbreviation	Name
A	0.98706966	O-M	optimal-method
A	0.96631115	O-C	optimal-class
B	0.89106897	AJ-MM	JUPTA-additional-methodcoverage-method
B	0.88396264	AD-MM	dynamic-additional-methodcoverage-method
B	0.86903655	AD-SM	dynamic-additional-statementcoverage-method
B	0.84982414	AJ-SM	JUPTA-additional-statementcoverage-method
B	0.83847218	AJ-MC	JUPTA-additional-methodcoverage-class
C	0.75402782	AD-MC	dynamic-additional-methodcoverage-class
C	0.73406138	AD-SC	dynamic-additional-statementcoverage-class
C	0.72864126	AJ-SC	JUPTA-additional-statementcoverage-class
D	0.61786046	TD-SM	dynamic-total-statementcoverage-method
D	0.61298966	TD-SC	dynamic-total-statementcoverage-class
D	0.60884759	TD-MM	dynamic-total-methodcoverage-method
D	0.60519989	R-M	random-method
D	0.60343253	TD-MC	dynamic-total-methodcoverage-class
D E	0.57437701	TJ-SC	JUPTA-total-statementcoverage-class
D E	0.57085862	TJ-MC	JUPTA-total-methodcoverage-class
D E	0.56503954	R-C	random-class
D E	0.56241115	TJ-MM	JUPTA-total-methodcoverage-method
E	0.52439724	TJ-SM	JUPTA-total-statementcoverage-method
F	0.44498506	U-C	untreated-class
F	0.41711287	U-M	untreated-method

TABLE 14
Tukey HSD Posthoc Test, ant, Mutation Faults

Grouping	Means	Techniques	
		Abbreviation	Name
A	0.99631087	O-M	optimal-method
A	0.98441287	O-C	optimal-class
A B	0.95430206	AD-SM	dynamic-additional-statementcoverage-method
B C	0.92111756	AD-MM	dynamic-additional-methodcoverage-method
C	0.88440038	AD-SC	dynamic-additional-statementcoverage-class
D	0.82878731	AD-MC	dynamic-additional-methodcoverage-class
D E	0.79314819	TD-SC	dynamic-total-statementcoverage-class
D E	0.79247656	TJ-SM	JUPTA-total-statementcoverage-method
D E	0.79149400	TD-SM	dynamic-total-statementcoverage-method
D E	0.79083175	TD-MC	dynamic-total-methodcoverage-class
D E	0.78620413	TD-MM	dynamic-total-methodcoverage-method
E F	0.76111431	R-M	random-method
E F	0.75905075	TJ-MM	JUPTA-total-methodcoverage-method
E F	0.74895869	AJ-SM	JUPTA-additional-statementcoverage-method
E F	0.74474450	AJ-MM	JUPTA-additional-methodcoverage-method
F	0.72146381	TJ-SC	JUPTA-total-statementcoverage-class
G	0.66871044	TJ-MC	JUPTA-total-methodcoverage-class
G H	0.65178044	R-C	random-class
H	0.61539313	AJ-SC	JUPTA-additional-statementcoverage-class
H	0.61119331	AJ-MC	JUPTA-additional-methodcoverage-class
H	0.60584538	U-C	untreated-class
I	0.52932588	U-M	untreated-method

effectiveness. Effectiveness is not the only factor driving the choice of a prioritization technique—the cost of the technique also matters. In our study, where various prototype implementations of techniques are utilized, we cannot fairly compare technique costs with precision. However, we do provide the following discussion of execution times.

As noted earlier, the worst-case time complexities of the JUPTA algorithms and the dynamic coverage-based prioritization algorithms are equivalent, but the former algorithms incur extra costs associated with call graph construction, and the latter algorithms incur extra costs due to the costs of collecting coverage information. Table 16 shows these extra

costs for JUPTA and the dynamic techniques for each program. In this table, the second column shows the time used to construct call graphs using our CHA analysis tool, and the following columns show the execution times for the program versions with method-level instrumentation, with statement-level instrumentation, and without instrumentation. As the table shows, the execution time with instrumentation (at both the method-level and the statement-level) is almost always larger, and sometimes much larger, than the sum of the execution time without instrumentation plus the time required to construct the call graph (the only exceptions being on jmeter versions 3 and 5).

TABLE 15
Tukey HSD Posthoc Test, All Programs, Mutation Faults

Grouping	Means	Techniques	
		Abbreviation	Name
A	0.98818677	O-M	optimal-method
A B	0.96194078	O-C	optimal-class
A B C	0.92289700	AD-SM	dynamic-additional-statementcoverage-method
A B C	0.90537695	AD-MM	dynamic-additional-methodcoverage-method
B C D	0.83453058	AD-SC	dynamic-additional-statementcoverage-class
C D E	0.81905986	AD-MC	dynamic-additional-methodcoverage-class
C D E F	0.79283388	AJ-SM	JUPTA-additional-statementcoverage-method
C D E F	0.79055822	AJ-MM	JUPTA-additional-methodcoverage-method
D E F G	0.73884906	TD-SM	dynamic-total-statementcoverage-method
D E F G	0.73593526	TD-MM	dynamic-total-methodcoverage-method
D E F G	0.72653351	TD-SC	dynamic-total-statementcoverage-class
D E F G	0.71982075	R-M	random-method
D E F G	0.71809390	TD-MC	dynamic-total-methodcoverage-class
D E F G	0.71242616	TJ-SM	JUPTA-total-statementcoverage-method
D E F G	0.69935194	TJ-MM	JUPTA-total-methodcoverage-method
E F G	0.69821968	AJ-SC	JUPTA-additional-statementcoverage-class
E F G	0.69177813	TJ-SC	JUPTA-total-statementcoverage-class
F G	0.67236983	AJ-MC	JUPTA-additional-methodcoverage-class
G H	0.632544543	TJ-MC	JUPTA-total-methodcoverage-class
G H	0.62107046	R-C	random-class
H	0.53136888	U-M	untreated-class
H	0.50109444	U-C	untreated-method

TABLE 16
Comparison of Time Costs (ms)

Object	Call graph construction	Method-level instrumentation	Statement-level instrumentation	Original execution
jtopas-v1	46	4,074	17,191	624
jtopas-v2	32	4,092	17,222	608
jtopas-v3	93	11,945	33,836	5,008
xmlsec-v1	234	2,274	2,808	2,137
xmlsec-v2	235	7,175	10,998	4,961
xmlsec-v3	203	7,526	13,089	6,614
jmeter-v1	251	11,275	11,294	10,904
jmeter-v2	234	11,246	11,357	10,935
jmeter-v3	608	11,388	11,528	11,045
jmeter-v4	344	11,457	11,654	11,061
jmeter-v5	422	11,428	11,622	11,092
ant-v1	327	19,589	20,358	19,157
ant-v2	376	36,245	36,566	34,539
ant-v3	381	34,382	35,459	33,743
ant-v4	624	108,919	117,037	104,332
ant-v5	593	145,903	154,618	142,875
ant-v6	656	145,302	154,976	142,886
ant-v7	843	199,212	242,331	188,167
ant-v8	811	197,445	245,185	187,481

5 RELATED WORK

Test case prioritization has been extensively studied. Here, we summarize existing research in two categories based on whether or not the techniques involved require dynamic coverage-based information.

5.1 Prioritization with Dynamic Coverage Data

Most existing test case prioritization techniques use dynamically collected test coverage information. Wong et al. [7] take advantage of source code changes between versions and previous execution traces, and then prioritize test cases based on “increasing cost per additional coverage” [7]. Rothermel et al. [1], [8] present a family of prioritization techniques at the statement level and provide the metric APFD for assessing the rate of fault detection. Through empirical studies, they show that these techniques (even the least sophisticated ones) can improve the effectiveness of regression testing. Later, Elbaum et al. [13] consider prioritizing test cases based on coarser-granularity information (i.e., function level coverage information). Taking into account test costs and fault severity, Elbaum et al. [22] propose a new metric, APFDc, which incorporates test costs and fault severity in the assessment of prioritization techniques. Jones and Harrold [5] propose prioritization techniques based on modified coverage/decision coverage. Jeffrey and Gupta [21] focus on executed statements and branches that affect or potentially affect test case output through relevant slices, and present a test case prioritization technique based on relevant slices.

There has also been research on prioritizing test cases using coverage information under limited time constraints [23], [24], [25]. Kim and Porter [23] study the situation in which resource constraints do not allow the execution of the whole test suite, and propose a technique utilizing the performance of test cases (e.g., coverage information and fault detection) in previous testing to prioritize test cases. Walcott et al. [25] consider an explicit time budget and propose a prioritization approach based on genetic algorithms. Walcott et al. also empirically compare the proposed approach using method coverage and block coverage with the untreated ordering, the reverse ordering, random

ordering, and a fault-aware prioritization technique. Zhang et al. [24] use integer linear programming in time-aware test-case prioritization, and they also compare four techniques using their approach with two techniques based on genetic algorithms and four traditional techniques for test case prioritization. Do et al. [39] consider the effects of time constraints on test case prioritization and find that such constraints substantially alter technique performance.

Before applying dynamic coverage-based techniques such as those investigated in the foregoing papers, testers must instrument code and collect coverage information for their test cases. As noted in Section 1, this can have drawbacks. JUPA does not require such instrumentation.

5.2 Prioritization without Dynamic Coverage Data

There has also been some prior research on techniques that do not require dynamic coverage-based information. Srikanth et al. [40] propose a test case prioritization approach based on combinations of several factors: customer-assigned priority, requirements volatility, implementation complexity, and fault proneness of the requirements. This approach identifies the importance of requirements by using weights assigned by users. Tonella et al. [41] take relative priority information from users and integrate it with multiple prioritization indices, and then they use machine learning algorithms to improve the prioritization result. Korel et al. [42] prioritize test cases based on Extended Finite State Machine (EFSM) system models. Ma and Zhao [43] distinguish fault severity using both users’ knowledge and program information, and attempt to detect severe faults first. Bryce et al. [44] present a model with which to define generic prioritization criteria for GUI and web applications in event-driven software.

While all of these techniques operate in cases where dynamic coverage information is not available, they all require user involvement or models that may not be accessible or available. Most of them also assume that users are familiar with the program under test and that their knowledge is accurate. In contrast, JUPA [17] is an automated approach. However, we conjecture that where such models or user involvement are possible, they can be

integrated with the JUPITA approach to provide more effective prioritization results.

6 CONCLUSION

Although test case prioritization has been extensively studied, most test case prioritization techniques proposed to date are based on the use of dynamic execution to capture code coverage information. In this paper, we proposed JUPITA, a test case prioritization approach for use on systems tested with JUnit test cases that functions in the absence of dynamic coverage data, operating instead on estimated coverage information collected through static call graph analysis of test cases. By avoiding the need to instrument code and execute test cases, our approach may be more applicable than dynamic coverage-based approaches in cases where gathering coverage information is inappropriate or is not cost effective.

To explore the cost benefits of JUPITA, we conducted an empirical study and compared several variants of JUPITA with several dynamic coverage-based prioritization techniques. Although dynamic coverage-based approaches retain effectiveness advantages, JUPITA does attain results surprisingly close, in terms of fault-detection effectiveness, to those techniques, and better than several of those techniques. Two JUPITA techniques in particular—those using feedback and operating at the method and statement coverage levels, exhibit the greatest effectiveness.

Because JUPITA is often less effective than the optimal approach, which produces ideal permutations of a JUnit test suite, we intend to attempt to improve JUPITA's effectiveness. Further, in addition to its application to regression testing, our approach can also be applied to any JUnit test suite in traditional testing processes to order test cases for their initial execution on a system, and the approach can be adapted to other forms of systems where test cases are implemented as code that calls tested subsystems. Both of these approaches could be worth further investigation.

ACKNOWLEDGMENTS

This work is supported by the National Basic Research Program of China under Grant No. 2009CB320703, the Science Fund for Creative Research Groups of China under Grant No. 61121063, and the National Natural Science Foundation of China under Grant Nos. 91118004 and 60803012. This work has also been supported by the US National Science Foundation (NSF) under grant CNS-0454203, and by the World Class University program under the National Research Foundation of Korea and funded by the Ministry of Education, Science and Technology of Korea (Project No. R31-30007).

REFERENCES

- [1] G. Rothermel, R. Untch, C. Chu, and M. Harrold, "Test Case Prioritization: An Empirical Study," *Proc. Int'l Conf. Software Maintenance*, pp. 179-188, 1999.
- [2] B. Beizer, *Software Testing Techniques*. Van Nostrand Reinhold, 1990.
- [3] H. Leung and L. White, "Insights into Regression Testing," *Proc. Int'l Conf. Software Maintenance*, pp. 60-69, 1989.
- [4] H. Do, G. Rothermel, and A. Kinner, "Empirical Studies of Test Cases Prioritization in a JUnit Testing Environment," *Empirical Software Eng.*, vol. 11, pp. 33-70, 2006.
- [5] J.A. Jones and M.J. Harrold, "Test-Suite Reduction and Prioritization for Modified Condition/Decision Coverage," *IEEE Trans. Software Eng.*, vol. 29, no. 3, pp. 195-209, Mar. 2003.
- [6] A. Srivastava and J. Thiagarajan, "Effectively Prioritizing Tests in Development Environment," *Proc. Int'l Symp. Software Testing and Analysis*, pp. 97-106, July 2002.
- [7] W. Wong, J. Horgan, S. London, H. Agrawal, and M. Bellcore, "A Study of Effective Regression Testing in Practice," *Proc. Int'l Symp. Software Reliability Eng.*, pp. 264-274, 1997.
- [8] G. Rothermel, R.H. Untch, C. Chu, and M.J. Harrold, "Prioritizing Test Cases for Regression Testing," *IEEE Trans. Software Eng.*, vol. 27, no. 10, pp. 929-948, Oct. 2001.
- [9] T. Ball and J.R. Larus, "Efficient Path Profiling," *Proc. Int'l Symp. Microarchitecture*, pp. 46-57, 1996.
- [10] M. Arnold and B.G. Ryder, "A Framework for Reducing the Cost of Instrumented Code," *Proc. SIGPLAN Conf. Programming Language Design and Implementation*, pp. 168-179, 2001.
- [11] M.M. Tikir and J.K. Hollingsworth, "Efficient Instrumentation for Code Coverage Testing," *Proc. SIGPLAN Int'l Symp. Software Testing and Analysis*, pp. 86-96, 2002.
- [12] D. Jeffrey and N. Gupta, "Improving Fault Detection Capability by Selectively Retaining Test Cases during Test Suite Reduction," *IEEE Trans. Software Eng.*, vol. 33, no. 2, pp. 108-123, Feb. 2007.
- [13] S. Elbaum, A.G. Malishevsky, and G. Rothermel, "Test Case Prioritization: A Family of Empirical Studies," *IEEE Trans. Software Eng.*, vol. 28, no. 2, pp. 159-182, Feb. 2002.
- [14] S. Elbaum, D. Gable, and G. Rothermel, "The Impact of Software Evolution on Code Coverage," *Proc. Int'l Conf. Software Maintenance*, pp. 169-179, 2001.
- [15] P.K. Chittimalli and M.J. Harrold, "Re-Computing Coverage Information to Assist Regression Testing," *Proc. Int'l Conf. Software Maintenance*, pp. 164-173, 2007.
- [16] T. Apiwattanapong, R. Santelices, P.K. Chittimalli, A. Orso, and M.J. Harrold, "MaTRIX: Maintenance-Oriented Testing Requirement Identifier and Examiner," *Proc. Testing: Academic and Industrial Conf.*, pp. 137-146, 2006.
- [17] L. Zhang, J. Zhou, D. Hao, L. Zhang, and H. Mei, "Prioritizing JUnit Test Cases in Absence of Coverage Information," *Proc. Int'l Conf. Software Maintenance*, pp. 19-28, 2009.
- [18] M. Renieris and S.P. Reiss, "Fault Localization with Nearest Neighbor Queries," *Proc. Int'l Conf. Automated Software Eng.*, pp. 30-39, 2003.
- [19] J.A. Jones, M.J. Harrold, and J. Stasko, "Visualization of Test Information to Assist Fault Localization," *Proc. Int'l Conf. Software Eng.*, pp. 467-477, May 2002.
- [20] D. Hao, T. Xie, L. Zhang, X. Wang, H. Mei, and J. Sun, "Test Input Reduction for Result Inspection to Facilitate Fault Localization," *J. Automated Software Eng.*, vol. 17, pp. 5-31, 2010.
- [21] D. Jeffrey and N. Gupta, "Test Case Prioritization Using Relevant Slices," *Proc. Int'l Computer Software Applications Conf.*, pp. 411-420, 2006.
- [22] S. Elbaum, A. Malishevsky, and G. Rothermel, "Incorporating Varying Test Costs and Fault Severities into Test Case Prioritization," *Proc. Int'l Conf. Software Eng.*, pp. 329-338, 2001.
- [23] J.-M. Kim and A. Porter, "A History-Based Test Prioritization Technique for Regression Testing in Resource Constrained Environments," *Proc. Int'l Conf. Software Eng.*, pp. 119-129, May 2002.
- [24] L. Zhang, S. Hou, C. Guo, T. Xie, and H. Mei, "Time-Aware Test-Case Prioritization Using Integer Linear Programming," *Proc. Int'l Symp. Software Testing and Analysis*, pp. 213-224, 2009.
- [25] K.R. Walcott, M.L. Soffa, G.M. Kaphammer, and R.S. Roos, "Time-Aware Test Suite Prioritization," *Proc. Int'l Symp. Software Testing and Analysis*, pp. 1-12, July 2006.
- [26] J. Dean, D. Grove, and C. Chambers, "Optimization of Object-Oriented Programs Using Static Class Hierarchy Analysis," *Proc. European Conf. Object-Oriented Programming*, pp. 77-101, 1995.
- [27] G.C. Murphy, D. Notkin, and E.S.-C. Lan, "An Empirical Study of Static Call Graph Extractors," *Proc. Int'l Conf. Software Eng.*, pp. 90-99, 1996.
- [28] O. Lhoták, "Comparing Call Graphs," *Proc. Seventh ACM SIGPLAN-SIGSOFT Workshop Program Analysis for Software Tools and Eng.*, pp. 37-42, 2007.
- [29] F. Tip and J. Palsberg, "Scalable Propagation Based Call Graph Construction Algorithms," *Proc. 15th ACM SIGPLAN Conf. Object-Oriented Programming Systems, Languages, and Applications*, pp. 281-293, 2000.

- [30] D.F. Bacon, "Fast and Effective Optimization of Statically Typed Object-Oriented Languages," PhD thesis, Computer Science Division, Univ. of California, Berkeley, 1997.
- [31] O. Shivers, "Control-Flow Analysis of Higher-Order Languages," PhD thesis, Carnegie Mellon Univ., 1991.
- [32] D. Grove, G. DeFouw, J. Dean, and C. Chambers, "Call Graph Construction in Object-Oriented Languages," *Proc. 12th ACM SIGPLAN Conf. Object-Oriented Programming Systems, Languages, and Applications*, pp. 108-124, 1997.
- [33] D.F. Bacon and P.F. Sweeney, "Fast Static Analysis of C++ Virtual Function Calls," *Proc. 11th ACM SIGPLAN Conf. Object-Oriented Programming Systems, Languages, and Applications*, pp. 324-341, 1996.
- [34] H. Do, S. Elbaum, and G. Rothermel, "Supporting Controlled Experimentation with Testing Techniques: An Infrastructure and Its Potential Impact," *Empirical Software Eng.*, vol. 10, no. 4, pp. 405-435, 2005.
- [35] H. Do and G. Rothermel, "A Controlled Experiment Assessing Test Case Prioritization Techniques via Mutation Faults," *Proc. Int'l Conf. Software Maintenance*, pp. 411-420, 2005.
- [36] Y. Ma, J. Offutt, and Y. Kwon, "MuJava: An Automated Class Mutation System," *J. Software Testing, Verification and Reliability*, vol. 15, no. 2, pp. 97-133, 2005.
- [37] H. Do and G. Rothermel, "On the Use of Mutation Faults in Empirical Assessments of Test Case Prioritization Techniques," *IEEE Trans. Software Eng.*, vol. 32, no. 9, pp. 733-752, Sept. 2006.
- [38] H. Do and G. Rothermel, "Using Sensitivity Analysis to Create Simplified Economic Models for Regression Testing," *Proc. Int'l Symp. Software Testing and Analysis*, pp. 51-62, 2008.
- [39] H. Do, S. Mirarab, L. Tahvildari, and G. Rothermel, "The Effects of Time Constraints on Test Case Prioritization: A Series of Controlled Experiments," *IEEE Trans. Software Eng.*, vol. 36, no. 5, pp. 593-617, Sept./Oct. 2010.
- [40] H. Srikanth, L. Williams, and J. Osborne, "System Test Case Prioritization of New and Regression Test Cases," *Proc. Int'l Symp. Empirical Software Eng.*, pp. 64-73, 2005.
- [41] P. Tonella, P. Avesani, and A. Susi, "Using the Case-Based Ranking Methodology for Test Case Prioritization," *Proc. Int'l Conf. Software Maintenance*, pp. 123-133, 2006.
- [42] B. Korel, G. Koutsogiannakis, and L. Tahat, "Application of System Models in Regression Test Suite Prioritization," *Proc. Int'l Conf. Software Maintenance*, pp. 247-256, 2008.
- [43] Z. Ma and J. Zhao, "Test Case Prioritization Based on Analysis of Program Structure," *Proc. Asia-Pacific Software Eng. Conf.*, pp. 471-478, 2008.
- [44] R. Bryce, S. Sampath, and A. Memon, "Developing a Single Model and Test Prioritization Strategies for Event-Driven Software" *IEEE Trans. Software Eng.*, vol. 37, no. 1, pp. 48-64, Jan./Feb. 2011.



Hong Mei received the PhD degree in computer science from Shanghai Jiaotong University in 1992. He is a professor at the School of Electronics Engineering and Computer Science, Peking University, PR China. He was a program cochair of many software conferences. He also serves on the editorial board of the *IEEE Transactions on Service Computing* and the *International Journal of Web Services Research*. His current research interests include software

engineering and software engineering environment, software reuse and software component technology, distributed object technology, and programming language. He is a senior member of the IEEE.



Dan Hao received the BS degree in computer science from the Harbin Institute of Technology in 2002 and the PhD degree in computer science from Peking University in 2008. She is an associate professor at the School of Electronics Engineering and Computer Science, Peking University. Her current research interests include software testing, debugging, and program comprehension.



languages. He is a student member of the IEEE.

Lingming Zhang received the BS degree in computer science from Nanjing University, China, and the MS degree in computer science from Peking University. He is currently working toward the PhD degree and is a research assistant in the Department of Electrical and Computer Engineering at The University of Texas at Austin. His research interests include regression testing, software evolution, mutation testing, software verification, and programming



on the editorial boards of the *Journal of Software Maintenance and Evolution: Research and Practice* and *Software Testing, Verification and Reliability*. His current research interests include software testing and analysis, program comprehension, software maintenance and evolution, software reuse and component-based software development, and service computing.

Ji Zhou received both the BS and MS degrees in computer science from Peking University. He is a software engineer at Baidu, Inc. His research interests include software testing and analysis.



Gregg Rothermel received the BA degree in philosophy from Reed College, the MS degree in computer science from SUNY Albany, and the PhD degree in computer science from Clemson University. He is currently a professor and the Jensen Chair of Software Engineering in the Department of Computer Science and Engineering, University of Nebraska-Lincoln. His research interests include software engineering and program analysis, with emphases on the application of program analysis techniques to problems in software maintenance and testing, and on empirical studies. He is an associate editor for the *IEEE Transactions on Software Engineering*. He served as program cochair for ICSE '07, program chair for ISSTA '04, and the chair of the steering committee for the International Conference on Software Maintenance. He is a member of the editorial boards for the *Empirical Software Engineering Journal* and the *Software Quality Journal*. He has served as a member of the program committees for a number of IEEE and ACM international conferences. He is a member of the IEEE Computer Society and the ACM.

of program analysis techniques to problems in software maintenance and testing, and on empirical studies. He is an associate editor for the *IEEE Transactions on Software Engineering*. He served as program cochair for ICSE '07, program chair for ISSTA '04, and the chair of the steering committee for the International Conference on Software Maintenance. He is a member of the editorial boards for the *Empirical Software Engineering Journal* and the *Software Quality Journal*. He has served as a member of the program committees for a number of IEEE and ACM international conferences. He is a member of the IEEE Computer Society and the ACM.